

SYSTEM ARCHITECTURE & ENGINEERING CONCEPT

High-Performance CAD & Schematic Generation Engine via TriMCP & NetBox

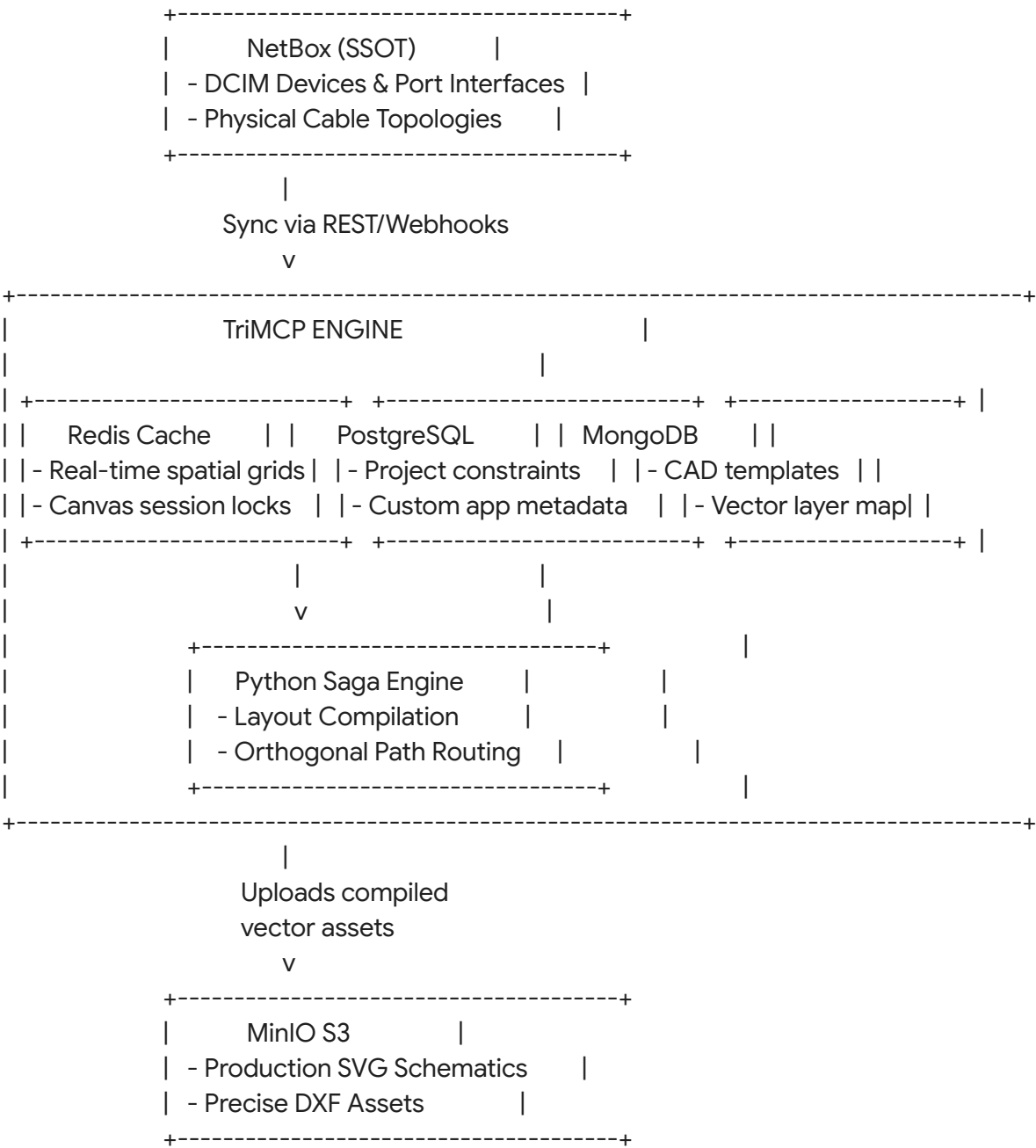
Target Architecture: TriMCP Quad-Stack Engine + NetBox Integration

Operational Scope: Automated Signal Flow, Cabling Schematics, and Rack Elevations

1. Executive Summary & Design Vision

The objective of this engineering initiative is to eliminate manual CAD/Vector drafting workflows for system cabling and rack layouts by treating drawings as **compiled data declarations**.

By leveraging your existing **NetBox** deployment as the absolute Source of Truth (SSOT) for physical network infrastructure and cable topologies, alongside your **TriMCP** advanced memory and transaction engine, we can build a dynamic, scalable compiler. This compiler reads topological relationships, calculates spatial coordinates on an infinite grid, and outputs fully standardized, layered, and vector-perfect vector graphics (SVG / DXF) stored directly in **MinIO**.



2. Platform Responsibilities

To achieve perfect logical and performance isolation, responsibilities are split with zero state-duplication:

A. NetBox (The Physical Model)

- Holds the canonical state of all IT/AV components, rack unit allocations, physical interfaces (ports), and actual cable connections.
- Exposes system state via the standardized [NetBox REST API](#) and Webhooks.

B. TriMCP (The Architectural compiler)

- **PostgreSQL:** Stores transactional project variables (customer metadata, sheet dimensions, custom drawing boundaries, override configurations).
- **MongoDB:** Stores presentation templates, custom icon graphics, vector representation layers for ports, and company title block configurations.
- **Redis:** Caches spatial lookup grids and transient connection coordinates to minimize DB lookups during active drawing compiles.
- **MinIO:** Acts as the high-availability vector vault for compiled drawing files (.svg, .dxf, .pdf).
- **Python Engine:** Acts as the compile engine, computing graph layout and routing algorithms.

3. Data-to-Drawing Compilation Pipelines

Drawing compilation is structured as an automated build pipeline triggered asynchronously by change events.

Pipeline A: Automated Rack Elevations

Automatically generates a physical layout vector of any given equipment rack modeled in NetBox.

[NetBox API Call] ---> [Map Height Units] ---> [Align coordinates] ---> [Inject Vector Art]
---> [MinIO Write]

1. **Extract:** Python fetches the target rack's device array from NetBox (/api/dcim/devices/?rack_id={id}).
2. **Calculate:**
 - The total height of the rack enclosure in Rack Units (H_{rack}) is converted to absolute spatial dimensions.
 - Our reference scale sets $1 \text{ RU} = 44.45 \text{ mm}$ (scaled to exactly 40px in the

vector coordinate space).

- For each device, the vertical Y -coordinate is calculated from its bottom-most occupied unit position (U_{position}):

$$Y_{\text{pos}}(U) = (H_{\text{rack}} - U_{\text{position}} - H_{\text{dev}} + 1) \times U_{\text{height}}$$

where H_{dev} is the device's physical height in Rack Units (U).

3. **Draw:** The compiler retrieves the device template style from MongoDB (e.g., standard color schemes, logo graphics) and outputs a precise SVG group element (<g>) positioned at coordinates $(X_{\text{rack_origin}}, Y_{\text{pos}})$.

Pipeline B: Automated Signal Flow Schematics

Uses a graph layout algorithm to map cable connections between hardware interfaces.

1. **Extract Graph Data:** Python queries NetBox for all cables and connected devices within a specified project scope or tenant:
 - **Nodes:** /api/dcim/devices/
 - **Edges (Connections):** /api/dcim/cables/
2. **Order Vertically & Horizontally (Sugiyama Algorithm):**
 - **Layering (X-Axis):** Devices are categorized into structural ranks based on the direction of network/signal flows (Inputs on the far left $\rightarrow$ Processing/Switching in the center $\rightarrow$ Outputs/Endpoints on the far right).
 - **Ordering (Y-Axis):** Nodes within the same layer are ordered to minimize cable path crossings.
3. **Port Mapping:** Device interface templates are queried from MongoDB. Individual interface connections (e.g., GigabitEthernet1/0/1, Dante Port 1) are aligned along the vertical left and right boundaries of their respective device blocks.
4. **Orthogonal Routing Algorithm:**

To render clean cabling lines that only bend at 90° angles without clipping equipment blocks, we construct a 2D grid pathfinding system.

Let the routing space be defined as a discrete grid matrix M . We represent any cable run C as a set of orthogonal line segments connecting source interface P_{src} to target interface P_{dst} . The routing algorithm must find path coordinates $W = [w_1, w_2, \dots, w_k]$ such that:

$$w_i \in M \quad \forall i$$

$$\Delta x_i \cdot \Delta y_i = 0 \quad \text{where } \Delta x_i = (x_{i+1} - x_i), \Delta y_i = (y_{i+1} - y_i)$$

- The engine applies a modified A* pathfinding algorithm or a channel routing algorithm that penalizes coordinate intersections and path bends, keeping lines separated by a minimum grid pitch of 8px .

4. Vector Output Standard (SVG Layer Matrix)

To ensure the compiled graphics are perfectly compatible with professional tools like CAD software and web interfaces, we compile SVG files using structured XML namespaces and layers.

```
<svg xmlns="[http://www.w3.org/2000/svg](http://www.w3.org/2000/svg)" viewBox="0 0 1920 1080" width="100%" height="100%">
  <defs>
    <!-- Standard Marker Definitions for Signal Flow Direction -->
    <marker id="arrow" viewBox="0 0 10 10" refX="6" refY="5" markerWidth="6" markerHeight="6"
orient="auto-start-reverse">
      <path d="M 0 1.5 L 10 5 L 0 8.5 z" fill="#4b5563" />
    </marker>
    <!-- Drop Shadow Filters for Hardware Blocks -->
    <filter id="box-shadow" x="-5%" y="-5%" width="110%" height="110%">
      <feDropShadow dx="0" dy="2" stdDeviation="3" flood-color="#000000"
flood-opacity="0.1"/>
    </filter>
  </defs>

  <!-- Layer 0: Drawing Sheet Frame & Metadata Title Block -->
  <g id="layer-sheet-frame" class="cad-layer">
    <rect x="10" y="10" width="1900" height="1060" fill="none" stroke="#e4e4e7"
stroke-width="2"/>
    <!-- Dynamic Title Block Text compiled directly from Postgres Metadata -->
    <text x="1600" y="1020" font-family="Inter, sans-serif" font-size="12" fill="#71717a">PROJECT:
System Layout Delta</text>
  </g>

  <!-- Layer 1: Device Rectangles & Port Text -->
  <g id="layer-devices" class="cad-layer">
    <!-- Managed Device Block -->
```

```

<g id="netbox-device-1042" transform="translate(100, 200)">
  <rect width="220" height="120" rx="4" fill="#ffffff" stroke="#18181b" stroke-width="2"
filter="url(#box-shadow)"/>
  <text x="15" y="30" font-family="Inter, sans-serif" font-size="14" font-weight="bold"
fill="#18181b">CORE SWITCH 01</text>
  <!-- Ports rendered programmatically -->
  <circle cx="220" cy="80" r="4" fill="#10b981" />
  <text x="210" y="84" font-family="Monospace" font-size="10" text-anchor="end"
fill="#4b5563">Gi1/0/1</text>
</g>
</g>

<!-- Layer 2: Orthogonal Cable Runs -->
<g id="layer-cabling" class="cad-layer">
  <!-- Orthogonal Cable Path representing NetBox Cable ID: 4509 -->
  <path d="M 320 280 L 450 280 L 450 420 L 580 420" fill="none" stroke="#2563eb"
stroke-width="2" marker-end="url(#arrow)"/>
  <text x="455" y="350" font-family="Inter, sans-serif" font-size="10" fill="#3b82f6"
transform="rotate(90, 455, 350)">CABLE-4509</text>
</g>
</svg>

```

5. Synchronization & Transaction Control (The TriMCP Saga Pattern)

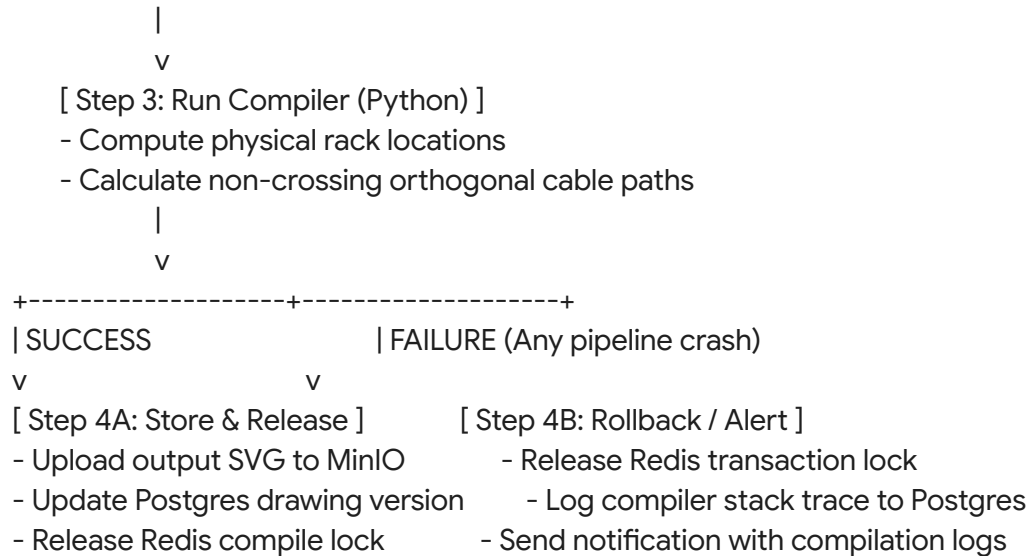
To keep your visual files synchronized with changes in NetBox without causing blocking delays, the system uses the **TriMCP Saga Pattern**.

This distributed transaction system ensures that NetBox mutations are fully validated, matched against database constraints in PostgreSQL, mapped into geometric modifications in MongoDB, and cleanly re-compiled to MinIO.

```

[ NetBox Webhook Triggered ]
  |
  v
[ Step 1: Initialize Saga Lock ]
- Redis flags target drawing ID as 'COMPILING'
  |
  v
[ Step 2: Extract & Map State ]
- Fetch structural data from NetBox REST API
- Fetch presentation templates from MongoDB

```



6. Optimization, Scaling & Guardrails

To run this platform at scale without overloading your systems, we put these protective guardrails in place:

- **Cache Compilation Paths in Redis:** Calculating complex orthogonal paths can be resource-heavy. We save these path coordinate maps directly to Redis. The compiler only recalculates paths if the underlying cable connections or device positions change in NetBox.
- **NetBox API Protection:** To avoid hitches or rate-limiting on NetBox from too many concurrent requests, TriMCP stores device templates locally in MongoDB. Instead of querying NetBox for every single port layout, it uses a local template lookup and only pulls unique operational connections.
- **Vector File Optimization:** The output SVGs use CSS style classes instead of writing raw style attributes inline on every single shape. This shrinks the resulting file sizes by up to **60%**, which reduces storage consumption in MinIO and makes them render much faster inside web browsers.

eof